

CS 301: Data Structures and Algorithms

Lecture 5: Binary Search Trees

Fall 2025 - Prof. Johnson

Learning Objectives:

- Understand binary search tree (BST) properties
- Implement BST insertion and deletion operations
- Analyze time complexity of BST operations
- Compare BSTs with balanced tree structures

1. Binary Search Tree Definition

A binary search tree is a binary tree data structure with the following properties:

- Left subtree contains only nodes with keys less than parent
- Right subtree contains only nodes with keys greater than parent
- Both subtrees must also be binary search trees

Example: BST Node Structure

```
class TreeNode:
    def __init__(self, val=0):
        self.val = val
        self.left = None
        self.right = None

def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

2. Time Complexity Analysis

The time complexity of BST operations depends on the tree's height:

Operation	Average Case	Worst Case
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

3. Practice Problems

Problem 1: Insert the following values into an empty BST: 50, 30, 70, 20, 40, 60, 80

Problem 2: What is the time complexity of finding the minimum element in a BST?

Problem 3: Implement an in-order traversal function for a BST

Next Lecture: AVL Trees and Tree Balancing

Reading: Chapter 12, sections 12.1-12.3